

## Data Engineering Exam.

**Instructions:** - Read each question carefully. - Select the best answer from the options provided. - Each question has only one correct answer.

---

### Question 1

Your ETL pipeline runs hourly processing files from S3, transforming and loading to database. Network failures cause retries. Pipeline runs twice on same file, creating duplicate records. How do you ensure idempotent processing?

**Options:**

- A) Distributed lock using Redis: acquire lock with file name before processing, release after completion, skip if lock exists.
- B) Database unique constraint on natural key columns preventing duplicate inserts, handle constraint violations gracefully in ETL code.
- C) Track processed files in metadata table: check if file processed before starting, insert record on success, skip processed files.
- D) Delete all target records for time period before loading to ensure clean slate and avoid accumulating duplicates.
- E) Configure retry logic with exponential backoff and max attempts, assume eventual success eliminates duplicate processing risk.

**Your Answer:** \_\_\_\_\_

### Question 2

Your DAG has 3 parallel tasks (A, B, C) feeding into task D. Task B fails at 2 AM due to an API timeout. Task D requires all upstream tasks to succeed. The orchestrator's retry mechanism successfully reruns B at 2:15 AM. What is the key orchestration feature being demonstrated?

**Options:**

- A) Dependency resolution with retry logic, where the orchestrator retries only the failed task while ensuring downstream tasks wait for all dependencies.
- B) Parallel execution only - tasks A, B, and C ran simultaneously to maximize throughput, demonstrating the orchestrator's ability to execute independent tasks concurrently.
- C) Simple scheduling - tasks ran at the specified 2 AM time, showing basic time-based triggering capabilities of the orchestration framework.
- D) Data storage - orchestrator stored the task results in its metadata database, enabling recovery and providing an audit trail of execution history.
- E) Automatic code correction - orchestrator detected the API timeout issue and automatically fixed the underlying code problem without human intervention.

**Your Answer:** \_\_\_\_\_

### Question 3

Your data quality monitoring shows: Accuracy 99%, Completeness 95%, Consistency 88%, Timeliness 99.9%, Validity 98%, Uniqueness 99%. Business users complain that “reports show different customer addresses depending on which dashboard they use.” Which dimension is causing the PRIMARY issue and requires immediate attention?

**Options:**

- A) Consistency (88%) - different values across systems leading to potential business incoherence.
- B) Accuracy (99%) - data contains errors where addresses have typos, wrong zip codes, or incorrect street numbers.
- C) Timeliness (99.9%) - data is too old and not being refreshed frequently enough to reflect recent customer address changes.
- D) Completeness (95%) - missing addresses where 5% of customer records have NULL or empty address fields.
- E) Validity (98%) - addresses don't match USPS formatting standards or contain invalid state codes and malformed zip codes.

**Your Answer:** \_\_\_\_\_

### Question 4

Your organization has 15 different data domains (sales, marketing, customer service, etc.), each with their own data teams. The central data platform team is overwhelmed managing all data pipelines and becoming a bottleneck. Which architecture pattern best addresses this scalability challenge?

**Options:**

- A) Data Mesh: decentralize data ownership to domain teams, treat data as products, enable self-service infrastructure with federated governance.
- B) Data Lake: centralize all raw data in a single repository with the platform team managing access and transformations for all domains.
- C) Data Warehouse: build a centralized enterprise data warehouse where the platform team models and integrates all domain data.
- D) Microservices: split the platform team into smaller specialized teams, each responsible for specific data pipeline technologies.
- E) Data Virtualization: create a unified virtual layer that the platform team manages to abstract all underlying domain data sources.

**Your Answer:** \_\_\_\_\_

### Question 5

Business initially wanted daily sales summary: date, product, store, total\_amount. Now wants hourly analysis. Existing fact table has daily grain. What's the best approach to add hourly capability?

**Options:**

- A) Add hour column to existing fact table, backfill with hour=0 for historical data, new data includes actual hour for granularity.
- B) Create new fact\_sales\_hourly at hourly grain, maintain both tables (daily aggregate, hourly detail) for different analytical needs.
- C) Rebuild fact table at hourly grain, drop daily table, all queries adjust to hourly grain with date-level aggregation in queries.
- D) Store hourly data in separate partition of same table, query optimizer determines whether to use daily or hourly partition automatically.
- E) Keep daily grain, add hour dimension with 24 rows (0-23), join fact to hour dimension for hourly reporting capability.

**Your Answer:** \_\_\_\_\_

### Question 6

You're storing 500TB of transaction data in HDFS with replication factor 3 across 50 nodes (10TB each). Node 23 fails permanently. What happens to data availability and storage capacity?

**Options:**

- A) All data on Node 23 is permanently lost and queries accessing that data will fail with errors requiring restore from backup.
- B) Data remains available due to replication; HDFS automatically begins re-replicating under-replicated blocks to maintain replication factor 3.
- C) System automatically fails over with no impact on capacity since HDFS dynamically reallocates the storage from the failed node to remaining nodes.
- D) HDFS requires manual intervention to restore access to data - an administrator must run fsck and manually redistribute blocks across the cluster.
- E) The entire cluster becomes unavailable because losing one node in a 50-node cluster breaks quorum requirements for the namenode.

**Your Answer:** \_\_\_\_\_

### Question 7

Your nightly ETL pipeline (10PM-2AM) extracts 5M rows, transforms, loads to warehouse. Sometimes completes in 2 hours, sometimes 6 hours, occasionally fails silently. What monitoring approach provides best visibility?

**Options:**

- A) Daily manual check of row counts comparing source vs destination to verify completeness of data loads.
- B) Database slow query log analysis to identify performance bottlenecks and optimize expensive SQL queries.
- C) Comprehensive observability: data quality metrics (row counts, nulls, schema), SLO alerts (>4hr runtime), data freshness checks.
- D) Email notification on pipeline failure sent by try-catch block wrapping the main ETL execution function.
- E) Weekly review of pipeline execution logs to identify patterns and trends in performance over time.

**Your Answer:** \_\_\_\_\_

### Question 8

Your ETL pipeline processes 1000 files/day, each taking 2-5 minutes. Currently runs on a t3.large EC2 instance (24/7, \$60/month). A colleague suggests serverless (Lambda/Cloud Functions). What is the PRIMARY consideration for this migration?

**Options:**

- A) Serverless is always cheaper than EC2 for any workload.
- B) Lambda has 15-minute execution limits.
- C) Serverless functions cannot access databases or external APIs.
- D) EC2 instances are more secure than serverless functions in all scenarios.
- E) You must rewrite the entire application in a different programming language for serverless.

**Your Answer:** \_\_\_\_\_

### Question 9

Your event stream processes user clicks (10M/day). Network issues cause duplicate events: same `user_id`, `page_id`, `timestamp` arrive 2-3 times. Simple deduplication on (`user_id`, `page_id`, `timestamp`) removes 30% as duplicates, but also removes legitimate rapid clicks. How do you improve deduplication accuracy?

**Options:**

- A) Keep all events without deduplication since determining true duplicates is impossible in practice.
- B) Increase deduplication window to 1-hour intervals to catch more potential duplicate events.
- C) Use composite key with tighter timestamp tolerance: deduplicate on (`user_id`, `page_id`, `timestamp_within_100ms`) plus `event_id` if available.
- D) Remove all events that have any duplicate attributes regardless of timing differences.
- E) Apply deduplication only at daily batch aggregation level rather than at ingestion time to maintain complete raw data for audit trails.

**Your Answer:** \_\_\_\_\_

### Question 10

Your sales dataset shows daily revenue: mostly \$50K-\$80K, but one day shows \$5M (100× normal). Investigation reveals it's a legitimate bulk order from enterprise client, not a data error. How should this be handled in preprocessing?

**Options:**

- A) Remove the outlier to maintain consistent statistical measures.
- B) Cap the value at 3 standard deviations from the mean to maintain statistical consistency.
- C) Flag as outlier but retain in dataset with metadata; handle appropriately based on analysis type (exclude from mean, include in total).
- D) Replace with the median value to avoid impact on downstream analysis and reporting.
- E) All outliers should be treated as data quality issues and removed during cleaning.

**Your Answer:** \_\_\_\_\_

### Question 11

You're aggregating daily sales by store. Store A has 10K transactions (avg \$50 each = \$500K total). Due to a bug, 5K transactions are duplicated in source data. Simple SUM(amount) reports \$750K (50% error). What aggregation approach prevents double-counting?

**Options:**

- A) Use SUM(amount) but divide result by expected duplication rate to estimate correct value.
- B) Deduplicate source data first on transaction\_id before aggregation to ensure each transaction counted once.
- C) Use AVG(amount) instead of SUM(amount) since averages are less sensitive to duplicates.
- D) Accept the error and add a footnote to reports about potential data quality issues.
- E) Sample data to estimate duplication rate and apply statistical correction to aggregates.

**Your Answer:** \_\_\_\_\_

### Question 12

Your downstream analytics team's dashboards break frequently because upstream source systems change schemas without warning: field renamed (user\_name → username), type changed (age: string → int), field removed (phone\_number deleted). What mechanism prevents these breaking changes?

**Options:**

- A) Downstream teams should monitor source systems daily and update code proactively when changes occur.
- B) Data contracts define schema, SLAs, and ownership; changes require approval and versioning with backward compatibility.
- C) Source systems have priority and can change at will; downstream systems must adapt to all changes.
- D) Implement error handling to catch schema mismatches and continue processing with default values.
- E) Store all source data as unstructured JSON to avoid schema validation issues entirely.

**Your Answer:** \_\_\_\_\_

### Question 13

IoT sensors send temperature readings every 10 seconds (8,640 readings/day per sensor, 1,000 sensors). Database has 3.15B rows (1 year). Queries for daily averages timeout. What preprocessing optimizes query performance?

**Options:**

- A) Add database index on timestamp and sensor\_id columns to speed up filtering and grouping operations.
- B) Partition table by month and create covering index including timestamp, sensor\_id, and temperature for queries.
- C) Increase database server memory and CPU to handle larger aggregations and improve query execution speed.
- D) Keep raw 10-second data, create materialized views with pre-aggregated hourly/daily averages that update incrementally on new data.
- E) Pre-aggregate during ingestion: compute rolling averages in stream processor, store only aggregated values, discard raw data.

**Your Answer:** \_\_\_\_\_

### Question 14

Your retail data warehouse has a sales fact table with grain: “one row per transaction line item” (OrderID, ProductID, LineItemID). Business wants a new metric: “average transaction value.” An engineer proposes changing the grain to “one row per transaction” (OrderID only), aggregating line items. What is the PRIMARY risk of this change?

**Options:**

- A) Query performance will improve significantly because aggregating at transaction level reduces the number of rows, making all queries faster.
- B) Storage costs will decrease substantially since aggregating line items into transactions can reduce table size by 70-80% on average.
- C) Grain change breaks existing queries and loses ability to analyze at line-item level, requiring a separate fact table instead.
- D) Change is fine and should be implemented immediately.
- E) Fact tables cannot have different grains according to Kimball methodology - every fact table must use the same level of detail.

**Your Answer:** \_\_\_\_\_

### Question 15

Your customer dataset has 30% missing values in “income” field. Which imputation strategy is MOST appropriate when income correlates strongly with other features (age, occupation, zip\_code) and will be used in predictive modeling?

**Options:**

- A) Delete all rows with missing income values to ensure only complete data is used.
- B) Replace all missing values with the global mean income across the entire dataset.
- C) Use predictive imputation: train ML model on complete records to predict missing income based on age, occupation, zip\_code.
- D) Replace missing values with zero to indicate unknown income level in the dataset.
- E) Missing values cannot be imputed accurately and should always be left as NULL.

**Your Answer:** \_\_\_\_\_

### Question 16

Your CTO proposes a multi-cloud strategy: “use best services from each provider (AWS S3, GCP BigQuery, Azure Synapse) to avoid vendor lock-in.” What is the PRIMARY hidden cost of this approach?

**Options:**

- A) Multi-cloud eliminates all vendor lock-in risks with no downsides.
- B) Cloud providers block multi-cloud architectures for competitive reasons.
- C) Operational complexity: maintaining expertise across multiple providers, managing different IAM systems, monitoring tools, and paying inter-cloud data transfer costs (\$0.08-0.12/GB).
- D) Multi-cloud always costs less than single-cloud because you can choose the cheapest service for each use case.
- E) Technical incompatibility - services from different providers cannot communicate with each other without extensive custom integration work.

**Your Answer:** \_\_\_\_\_



## Question 17

Your SCD Type 2 dim\_customer has 10M rows (5M unique customers  $\times$  2 average versions). Daily ETL checks all 5M customers for changes using LEFT JOIN to detect new/modified records. Query takes 45 minutes scanning full dimension. What optimization speeds this up?

### Options:

- A) Add indexes on natural keys (customer\_id) and is\_current flag to speed up lookup and filtering operations.
- B) Hash comparison: compute hash of customer attributes, compare hashes instead of full record comparison for faster detection.
- C) Incremental detection: use change data capture (CDC) from source, only process customers with changes instead of checking all customers.
- D) Partitioning: partition dimension table by date ranges, query only recent partitions for change detection and comparison.
- E) Parallel processing: split 5M customers into 100 batches, process batches in parallel on Spark cluster for distributed execution.

**Your Answer:** \_\_\_\_\_

## Question 18

You are tasked with designing a dimensional model for a university's student enrollment system. The system needs to track:

- Student enrollments in courses.
- Course information (course name, department, credits).
- Student information (name, major, admission year).
- Instructor information.
- Semester information (year, term, start/end dates).
- Grades received.

**Design a Star Schema for this scenario. Your diagram should include:**

1. Fact table(s) - identify what to measure.
2. Dimension tables - identify the descriptive context.
3. Specify the key attributes in each table.
4. Show relationships between tables.
5. Indicate primary and foreign keys.

**Additionally, explain:** - Why you chose this particular fact table design. - What types of analytical queries this model would support well. - What business questions this model can answer.

**Business Questions Supported:** - How many students enrolled per semester? - What is the average grade by department? - Which instructors teach the most courses? - Enrollment trends by major over time - Course popularity analysis

### Options:

**Your Answer:** \_\_\_\_\_

### Question 19

Your fraud detection system currently runs hourly batch jobs scanning 500K transactions, identifying fraud 1-6 hours after occurrence (average \$5K loss per fraud case). Switching to stream processing would detect fraud within seconds but costs \$3K/month more in infrastructure. What is the PRIMARY decision factor?

**Options:**

- A) Always choose batch processing since it's simpler and more cost-effective than streaming.
- B) Stream processing is always superior to batch for any data volume or use case.
- C) Business value calculation: fraud loss reduction (earlier detection saves \$X) vs streaming infrastructure cost (\$3K/month).
- D) The latency requirement alone determines architecture - fraud detection needs real-time regardless of cost.
- E) Hybrid approach is impossible - systems must choose either batch or streaming exclusively.

**Your Answer:** \_\_\_\_\_

### Question 20

You're designing a data retention policy for a financial services company that must comply with GDPR, SOX, and industry regulations. Which data lifecycle phase requires the MOST complex policy decisions balancing legal, business, and technical requirements?

**Options:**

- A) Data collection and ingestion - requires consent management and privacy notices.
- B) Data processing and analysis - must ensure algorithms don't create bias.
- C) Data visualization and reporting - needs role-based access controls.
- D) Data archival, retention, and deletion - balances legal retention with deletion rights.
- E) Data replication and backup - must maintain security across all copies.

**Your Answer:** \_\_\_\_\_

## Question 21

You're cleaning customer data and find email addresses: "john@example.com", "JOHN@EXAMPLE.COM", " john@example.com ", "john@EXAMPLE.com". Your deduplication logic flags them as separate records. What is the MOST comprehensive approach to handle this?

### Options:

- A) Delete all records except the first one encountered to maintain data integrity and ensure only one version exists in the system.
- B) Keep all records since they might be different people sharing the same email address for a family account or business team.
- C) Deduplicate records to ensure consistency.
- D) Only remove leading/trailing spaces while preserving case sensitivity to maintain the user's original input format as entered.
- E) Normalize by converting to lowercase, trimming whitespace, validating email format, and intelligently deduplicating while preserving the most recent or complete record.

**Your Answer:** \_\_\_\_\_

## Question 22

Your ETL workflow has 20 tasks with complex dependencies: some run in parallel, others are sequential, some retry on failure, others need manual approval. Which orchestration pattern handles this complexity most effectively?

### Options:

- A) Cron jobs with shell scripts checking file markers for dependencies and email notifications for failures.
- B) Stored procedures in the database with job scheduling tables tracking execution state and dependencies.
- C) Simple Python script running tasks sequentially with try-catch blocks for error handling and logging.
- D) Message queue (RabbitMQ) where each task publishes messages to trigger dependent tasks in event-driven fashion.
- E) Workflow orchestrator (Airflow, Prefect, Dagster) with DAG definition, task dependencies, retries, and monitoring UI.

**Your Answer:** \_\_\_\_\_

### Question 23

Your organization is migrating from an on-premises Oracle database to a cloud data warehouse. The legacy ETL jobs take 6 hours to run nightly, transforming 2TB of data on a dedicated ETL server before loading. The new cloud data warehouse offers massively parallel processing. Which statement best describes the architectural decision you should make?

**Options:**

- A) Switch to ELT to leverage cloud warehouse compute.
- B) Keep ETL approach as it always provides better data quality control and allows for centralized transformation logic that can be audited before data reaches the warehouse.
- C) ELT is inappropriate for cloud data warehouses due to security concerns about storing raw, untransformed data that may contain sensitive information.
- D) Use ETL in cloud environments to maintain consistency with existing processes and reduce the learning curve for the team already trained on ETL patterns.
- E) The choice between ETL and ELT has no impact on performance or cost in cloud environments since both approaches ultimately load the same data.

**Your Answer:** \_\_\_\_\_

### Question 24

Your event logs grow 500GB/day. Regulations require 7-year retention but queries only access recent 90 days. Storage costs are \$100K/year and growing. What strategy optimizes cost while maintaining compliance?

**Options:**

- A) Delete data older than 90 days since it's never queried, accept compliance risk for cost savings and performance.
- B) Tiered storage: hot tier (recent 90 days) on fast SSD, warm tier (91-365 days) on HDD, cold tier (1-7 years) on Glacier with lifecycle policies.
- C) Compress old data using higher compression ratios: recent data with Snappy (fast), old data with ZSTD level 19 (maximum compression).
- D) Aggregate old data: keep raw data for 90 days, replace 91+ day data with daily summaries, discard raw details.
- E) Export old data to tape backup stored offsite, delete from production systems, retrieve from tape if needed for audits.

**Your Answer:** \_\_\_\_\_

## Question 25

Customer #12345 changed address on March 15, 2025. Your dimension has Type 2 SCD. On May 1, 2025, an analyst runs a report: “What was customer #12345’s address on March 1, 2025?” The query uses effective\_date filtering. What is the expected behavior?

### Options:

- A) Query returns NULL because the address changed and historical data is not maintained in standard dimension tables.
- B) Returns current address.
- C) Query returns the old address that was valid on March 1, 2025 (before the March 15 change).
- D) Query returns both old and new addresses in separate rows, requiring the analyst to manually determine which was valid.
- E) Type 2 SCD cannot answer historical questions because it only maintains the current state of dimensional attributes.

**Your Answer:** \_\_\_\_\_

## Question 26

Your event-driven system has 30 services communicating via Kafka. Service A publishes order.created events, Services B/C/D/E consume and process differently. How do you test end-to-end flows without running all 30 services?

### Options:

- A) Contract testing: each service defines event schema contracts, verify producers/consumers honor contracts independently.
- B) Integration tests: spin up all 30 services in Docker Compose, publish events, assert expected outcomes across services.
- C) Unit tests only: mock Kafka producer/consumer in each service, test business logic isolation without actual message passing.
- D) Manual testing: deploy to staging environment, manually trigger events, observe logs and outcomes across services.
- E) Production monitoring: skip pre-production testing, monitor production events, roll back if issues detected in real traffic.

**Your Answer:** \_\_\_\_\_

### Question 27

Your daily batch pipeline failed at 3 AM after processing 70% of records due to a transient network error. The pipeline performs: (1) DELETE existing records for today, (2) INSERT new records from source, (3) UPDATE aggregation tables. Without idempotency, what is the PRIMARY risk if you simply restart the pipeline?

**Options:**

- A) The pipeline will run slower on the second attempt because the database must check for existing records before inserting new ones.
- B) Network errors will occur again since the underlying infrastructure issue hasn't been resolved and the same network path will be used.
- C) Storage costs will increase marginally due to transaction logs and temporary tables created during the failed execution.
- D) Data corruption occurs because restarting will re-execute all steps, potentially deleting valid data and creating duplicate records.
- E) The pipeline will automatically skip completed steps using checkpoint recovery mechanisms built into modern ETL frameworks.

**Your Answer:** \_\_\_\_\_

### Question 28

Order service needs product pricing for checkout. Pricing service has 50ms API latency. Every order checks pricing for 5 products = 250ms overhead per order. Pricing rarely changes (hourly). How do you optimize this communication?

**Options:**

- A) Synchronous REST calls with connection pooling and HTTP keep-alive to reduce connection overhead and handshake time.
- B) GraphQL: single request fetching all 5 product prices in one query reducing round trips and network latency significantly.
- C) Event-driven cache: pricing service publishes price change events, order service maintains local cache, serves from memory instantly.
- D) gRPC with multiplexing: use gRPC for efficient binary protocol and HTTP/2 multiplexing for better performance than REST.
- E) Database sharing: order service queries pricing database directly bypassing API call overhead for fastest data access.

**Your Answer:** \_\_\_\_\_

### Question 29

A data breach exposed customer PII because a junior analyst had production database access they didn't need. Post-incident analysis reveals unclear data ownership, inconsistent access controls across systems, and no audit trail of data usage. Which data governance framework component would have MOST effectively prevented this breach?

**Options:**

- A) Implementing faster database query engines with optimized indexes to improve performance and reduce the time analysts spend waiting for query results.
- B) Creating more detailed ETL documentation with comprehensive data lineage diagrams that track all data transformations from source to destination.
- C) Implementing RBAC (Role-Based Access Control) with least privilege principles to ensure users only have access to the specific data and systems required for their job functions.
- D) Migrating to a cloud data warehouse with built-in encryption at rest and in transit, along with advanced threat detection capabilities.
- E) Increasing data backup frequency from daily to hourly to ensure minimal data loss in case of incidents.

**Your Answer:** \_\_\_\_\_

### Question 30

Streaming pipeline processes 10K events/second from IoT devices into data warehouse. Occasionally sensors send corrupt data (negative temperatures, impossible values) polluting warehouse. How do you enforce quality in real-time?

**Options:**

- A) Batch validation: warehouse materialized view with quality rules, flag bad data daily for cleanup and correction.
- B) Stream processing validation: Kafka Streams/Flink validates each event, routes valid to warehouse, invalid to DLQ for inspection.
- C) Database constraints: add CHECK constraints on warehouse tables rejecting invalid inserts at database layer for data integrity.
- D) Post-load cleanup: scheduled job queries warehouse for invalid data, deletes or corrects bad records nightly.
- E) Sampling inspection: validate 1% of events randomly, assume overall quality good if sample passes quality checks.

**Your Answer:** \_\_\_\_\_

### Question 31

You're modeling retail transactions. A business analyst wants to add "product\_name" and "customer\_email" directly to the fact table to "make queries simpler and avoid joins." What is the PRIMARY problem with this approach?

**Options:**

- A) Fact tables cannot store text fields for technical reasons related to database engine optimization.
- B) Massive redundancy, storage costs, and update complexity.
- C) SQL queries will run faster with descriptive attributes in fact tables because joins to dimension tables add query overhead.
- D) This is actually the recommended approach in dimensional modeling according to modern data warehouse optimization techniques.
- E) Fact tables can only store integers and floating-point numbers due to indexing requirements.

**Your Answer:** \_\_\_\_\_

### Question 32

Your Spark ETL job processes 100GB daily data on 10 r5.4xlarge instances (16 vCPU, 128GB RAM each) costing \$300/day. Job completes in 2 hours with 20% average CPU utilization. What optimization reduces cost most effectively?

**Options:**

- A) Switch to spot instances at 70% discount, accept occasional interruptions with automatic retry logic for cost savings.
- B) Right-size to smaller instances (r5.xlarge with 4 vCPU, 32GB RAM), use more instances to maintain throughput at lower cost.
- C) Optimize Spark configuration: increase parallelism, tune memory allocation, reduce shuffle overhead to improve resource utilization efficiency.
- D) Migrate to serverless (AWS Glue, Databricks Serverless) paying only for actual compute used during 2-hour execution window.
- E) Schedule job during reserved instance hours, purchase 1-year reserved instances for 40% discount on committed usage.

**Your Answer:** \_\_\_\_\_



### Question 33

According to the CAP theorem, a distributed system can only guarantee two of three properties simultaneously. What are these three properties?

**Options:**

- A) Cost, Availability, Performance.
- B) Consistency, Accuracy, Partition tolerance.
- C) Computing, Analytics, Processing.
- D) Capacity, Availability, Performance.
- E) Consistency, Availability, Partition tolerance.

**Your Answer:** \_\_\_\_\_

### Question 34

Your company has customer data in 8 systems: CRM (100K records), Billing (95K), Support (80K), Marketing (120K). Same customer appears with different emails, addresses, IDs across systems. Reports show “200K total customers” but actual unique count is 110K. What MDM (Master Data Management) approach solves this?

**Options:**

- A) Choose one system as source of truth and delete customer data from all other systems permanently.
- B) Golden record creation: match/merge records across systems, create authoritative customer master with links to source records.
- C) Accept data inconsistency as unavoidable when dealing with multiple operational systems.
- D) Store all customer data separately in each system without any integration or deduplication efforts.
- E) Manually reconcile customer records weekly by having staff compare data across all systems.

**Your Answer:** \_\_\_\_\_

### Question 35

Your daily ETL loads a 500GB customer table (10M rows) using full table extract-replace pattern. Load takes 4 hours, impacts source database performance, and violates nightly batch window (3 hours). Only 50K rows (~0.5%) change daily. What optimization reduces load time and source impact?

**Options:**

- A) Continue full loads but run more frequently (every 6 hours) to reduce per-load volume and impact.
- B) Reduce table size by archiving historical records to decrease the amount of data processed.
- C) Add more compute resources to the ETL process to handle the full load more quickly.
- D) Switch to incremental load: extract only new/changed rows using updated\_timestamp or CDC, reducing daily load to 50K rows.
- E) Compress the data during transfer to reduce network bandwidth and improve overall performance.

**Your Answer:** \_\_\_\_\_

### Question 36

Your team is deciding between managed Postgres (RDS/Cloud SQL) vs self-managed Postgres on EC2/VMs. The managed service costs 40% more. Which factor would MOST justify the higher cost of managed services?

**Options:**

- A) Managed services always provide better performance than self-managed databases.
- B) Operational overhead: automated backups, patching, HA failover, monitoring, and scaling free up team to focus on data engineering instead of database administration.
- C) Managed services eliminate all need for database expertise on the team.
- D) Self-managed databases cannot achieve high availability or disaster recovery.
- E) Managed services provide unlimited storage and compute resources at no additional cost.

**Your Answer:** \_\_\_\_\_

### Question 37

Your data pipeline runs successfully every night (no errors logged), but business users report incorrect numbers in dashboards. Investigation reveals: (1) Pipeline processes 0 records (source API returned empty), (2) No alerts triggered. What data quality check was missing?

**Options:**

- A) Code review process to catch bugs before deployment would have prevented this issue.
- B) More comprehensive error handling with try-catch blocks around every operation.
- C) Data volume anomaly detection: alert when record count deviates significantly from baseline (e.g., 0 vs expected 100K).
- D) Running the pipeline more frequently to catch issues faster wouldn't prevent silent failures.
- E) Adding more unit tests to the pipeline code to ensure functions work correctly.

**Your Answer:** \_\_\_\_\_

---

## EXAM04 - Answer Sheet

| Question | Answer    |
|----------|-----------|
| 1        | A B C D E |
| 2        | A B C D E |
| 3        | A B C D E |
| 4        | A B C D E |
| 5        | A B C D E |
| 6        | A B C D E |
| 7        | A B C D E |
| 8        | A B C D E |
| 9        | A B C D E |
| 10       | A B C D E |
| 11       | A B C D E |
| 12       | A B C D E |
| 13       | A B C D E |
| 14       | A B C D E |
| 15       | A B C D E |
| 16       | A B C D E |
| 17       | A B C D E |
| 18       | A B C D E |
| 19       | A B C D E |
| 20       | A B C D E |
| 21       | A B C D E |
| 22       | A B C D E |
| 23       | A B C D E |
| 24       | A B C D E |
| 25       | A B C D E |
| 26       | A B C D E |
| 27       | A B C D E |
| 28       | A B C D E |
| 29       | A B C D E |
| 30       | A B C D E |
| 31       | A B C D E |
| 32       | A B C D E |
| 33       | A B C D E |
| 34       | A B C D E |
| 35       | A B C D E |
| 36       | A B C D E |
| 37       | A B C D E |



